COMMANDS AND SET-UP

DVC

1. `dvc init` → Initialize DVC in the project (creates `.dvc/` and config).
2. `dvc remote add <name> <url>` → Add a remote storage location (S3, GDrive, MinIO, etc.).
3. `dvc add <file/dir>` → Start tracking a data file or directory with DVC.
4. `dvc commit` → Save current workspace data state to DVC cache (without re-running pipeline).
5. `dvc push` → Upload tracked data/artifacts from local cache to the remote.
6. `dvc remote list` → Show all configured DVC remotes.
7. `dvc remote remove <name>` → Delete a configured DVC remote.
8. `dvc config -l` → Display all DVC configuration settings (local + global).
9. `dvc repro` → Re-run the pipeline stages whose dependencies or outputs changed.
10. `dvc dag` → Visualize the DVC pipeline as a dependency graph.
11. `dvc exp run` → Run an experiment and track metrics/params without a Git commit.
12. `dvc exp apply <exp-name>` → Apply a specific experiment's results to the working directory.
13. `dvc exp remove <exp-name>` → Delete a saved experiment.
14. dvc exp diff <first exp name> <secnd exp name>
15. `dvc exp pull` → Download experiments from a remote repository.
16. dvc exp run --set-param train.lr=0.02
    i. dvc exp run --queue -S train.lr=0.1
    ii. dvc exp run --queue -S train.lr=0.001
    iii. dvc exp run --run-all

Correct way: queue each combination

You must run one queue command **per experiment**.

Example (3×3 grid = 9 experiments):

Auto (Lua)

```
dvc exp run --queue -S model_building.random_state=20 -S
model_building.n_estimators=10
dvc exp run --queue -S model_building.random_state=20 -S
model_building.n_estimators=12
dvc exp run --queue -S model_building.random_state=20 -S
model_building.n_estimators=13
```

```
dvc exp run --queue -S model_building.random_state=30 -S
model_building.n_estimators=10
dvc exp run --queue -S model_building.random_state=30 -S
model_building.n_estimators=12
dvc exp run --queue -S model_building.random_state=30 -S
model_building.n_estimators=13

dvc exp run --queue -S model_building.random_state=40 -S
model_building.n_estimators=10
dvc exp run --queue -S model_building.random_state=40 -S
model_building.n_estimators=12
dvc exp run --queue -S model_building.random_state=40 -S
model_building.n_estimators=13
```

Then execute them:

dvc queue status ; to see all the exp queued

Auto

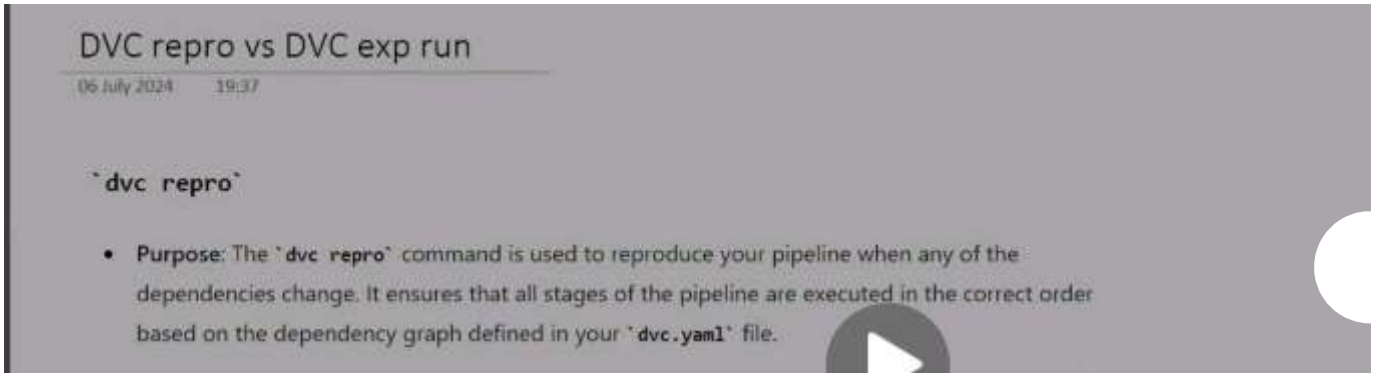
```
dvc exp run --run-all
```

16 dvc exp show

dvc exp run vs dvc repro

dvc repro run the pieplines but does not trcks the experiment

dvc exp run the pipelines and also tracks the experiment



- Use Cases:
  1. Regular Pipeline Execution: Use `dvc repro` when you want to run your pipeline as defined, typically after changing code, input data, or configuration files (like `params.yaml`).
  2. Reproducing Results: Use it to reproduce results or ensure that your pipeline produces consistent outputs after making changes.

`dvc exp run`

- Purpose: The `dvc exp run` command is designed for running and managing experiments. It allows you to tweak parameters, run different configurations, and keep track of multiple experimental results without altering your main pipeline.

SETTING UP MIN IO FOR DVC

In case of pipline u dont have to add : dvc add <File to be track> , it is intellgent enough to see and tracj all outputs of dvc.yaml

in case of piplines , it creates dvc.lock file instead of dvc file where data etc hash is present , instead it maintains dvc.lock which has all the information of track things. . It is also called state file

1).\minio.exe server D:\minio-data --address ":9100" --console-address ":9101"

- MinIO uses **two ports** for different purposes.
- **API Port (9100)** → Used by tools like **DVC, AWS CLI, Python** to upload/download data.
- **Console Port (9101)** → Used only in **browser** for UI (buckets, users, settings).
- Your config used **9101**, which is **wrong for DVC**.
- DVC sends **S3 API requests**, which only work on **9100**.
- Console port **does not understand API calls**, so it returned **400 Bad Request**.
- Changing the endpoint to **9100 fixed the issue**.

Key Rule to Remember ✅

- **Apps & tools → API port (9100)**
- **Browser → Console port (9101)**

2)dvc remote add -d <remote name> s3://<minio bucket name>

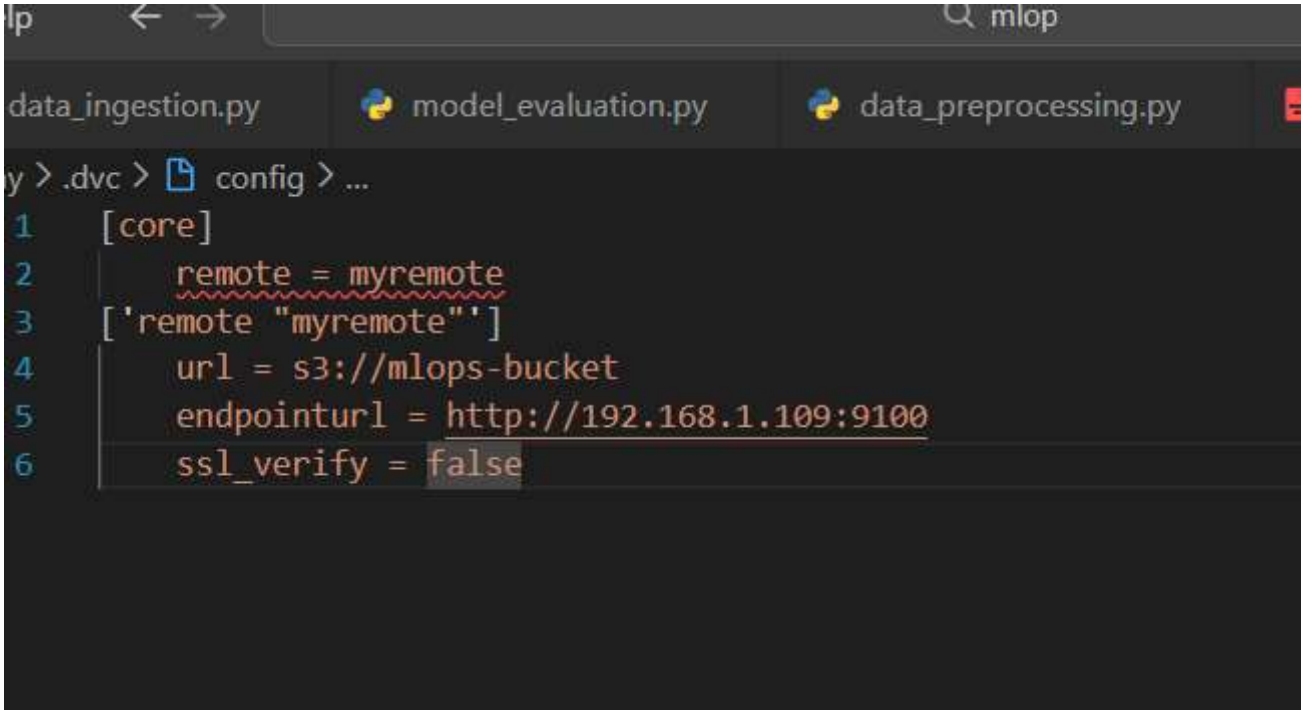
dvc remote add -d myremote s3://mlops-bucket

3)General DVC Remote Configuration Structure

Auto (Bash)

```
[core]
    remote = myremote                    # Default remote to use

['remote "myremote"']                    # Remote name (can be anything)
    url = s3://bucket-name               # Where to store data
    endpointurl = http://ip:port         # Server address
    access_key_id = your_key             # Username/access key
    secret_access_key = your_secret      # Password/secret
    ssl_verify = false                   # SSL certificate check
```



dvc commit

dvc push

HOW TO WRITE DVC.YAML FILES

> `dvc.yaml` **is a dependency graph for your ML pipeline**

It tells DVC:

- **HOW to run** each step
- **WHAT files it depends on**
- **WHAT files it produces**

If data is stored in sub-folder , tell about it complte

for example:

1) Data / Raw / Train , Data/Raw /Test

then mentioned both

2) if only file in side the folder mentioed in too , it that is created as output

Models/model.pkl

3. In src dont use os.path.abspath(..) to move into sub folder , it will work inside src but when u wll do dvc repro in main folder , so dvc will run pipline when it will find .. it will go out of the main folder in which u working so error will be thrown as it will not be able to find output files.use os.getcwd() ,it wll give you your root dire withouting moving into .. like that
4. dvc.yaml sees params.yaml file for parameter checking , so name it params.yaml not parameters.yaml
5. when telling parameters inside dvc.yaml , the names of parametrs should me mentioned as in params.yaml : <key name as in params.yml>.<param name as in yaml>
6. Data folder , Model folder ,score folder: in short your artifcats folder should not be tracked by the git

First: What `dvc.yaml` REALLY is (1 line)

> `dvc.yaml` **is a dependency graph for your ML pipeline**

It tells DVC:

- **HOW to run** each step
- **WHAT files it depends on**
- **WHAT files it produces**

That's it.

🔴 Biggest Mistake Beginners Make

They think:

> " `dvc.yaml` is a config file like YAML params"

❌ NO

It is **more like a Makefile for ML**.

🧩 Core Rule (MEMORIZE THIS)

For **EVERY pipeline step**, ask 3 questions:

Auto (C#)

```
1 What command runs this step?
2 What files does it READ?
3) What parameters it uses from params.yml
4) What files does it CREATE?
```

Those map directly to:

	`cmd`
	`deps`
	`outs`

🧱 Basic Skeleton (ALWAYS SAME)

Auto (Visual Basic .NET)

```
stages:
  stage_name:
    cmd: python path/to/script.py
    params:
      - something
    deps:
      - something
    outs:
      - something
```

Nothing else is mandatory.

🧪 Let's Build ONE Stage Slowly (Your project)

Example: `data_` [ingestion.py]

Ask the 3 questions:

1️⃣ How do I run it?

Auto

```
python src/data_ingestion.py
```

➡ goes into `cmd`

2️⃣ What does it READ?

Look inside code:

Auto

```
pd.read_csv("data/raw/spam.csv")
```

➡ goes into `deps`

3️⃣ What does it CREATE?

Look inside code:

Auto

```
data/interim/train.csv
data/interim/test.csv
```

➡ goes into `outs`

✅ Final Stage

Auto (Bash)

```
stages:
  data_ingestion:
    cmd: python src/data_ingestion.py
    deps:
      - src/data_ingestion.py
      - data/raw/spam.csv
    outs:
      - data/interim
```

📌 Notice:

- We tracked **folder**, not individual files (BEST PRACTICE)

🧩 Add Second Stage (Chain Reaction)

`data_` [preprocessing.py]

Ask again:

- reads → `data/interim`
- creates → `data/processed`

Auto (Bash)

```
  data_preprocessing:
    cmd: python src/data_preprocessing.py
    deps:
      - src/data_preprocessing.py
      - data/interim
    outs:
      - data/processed
```

🔗 IMPORTANT: How DVC Understands ORDER

> There is **NO explicit order** in `dvc.yaml`

Order is inferred automatically:

Auto

```
data/raw → data/interim → data/processed
```

Because:

- `data_preprocessing` **depends on** output of `data_ingestion`

This is why `deps` and `outs` must be correct.

🧠 FULL PIPELINE (Your Case)

Auto (Bash)

```
stages:
  data_ingestion:
    cmd: python src/data_ingestion.py
    deps:
      - src/data_ingestion.py
    params:
      - data_ingestion.test_size
      - data_ingestion.random_state
    outs:
      - Data/Raw/Train
      - Data/Raw/Test

  data_preprocessing:
    cmd: python src/data_preprocessing.py
    deps:
      - src/data_preprocessing.py
      - Data/Raw/Train
      - Data/Raw/Test
    outs:
      - Data/Processed/Train
      - Data/Processed/Test
```

```
  feature_engineering:
    cmd: python src/feature_eng.py
    deps:
      - src/feature_eng.py
      - Data/Processed/Train
      - Data/Processed/Test
    params:
      - Feature_Eng.k
    outs:
      - Data/FeatureEng

  Model_Building:
    cmd: python src/model_building.py
    deps:
      - src/model_building.py
      - Data/FeatureEng
      - Data/Processed/Train
    params:
      - Model_Building.max_iter
      - Model_Building.penalty
      - Model_Building.C
      - Model_Building.solver
      - Model_Building.class_weight
      - Model_Building.random_state
      - Model_Building.n_jobs
    outs:
      - Models/logistic_model.pkl

  Model_Eval:
    cmd: python src/model_eval.py
    deps:
      - src/model_eval.py
      - Models/logistic_model.pkl
      - Data/FeatureEng
      - Data/Processed/Test
    outs:
      - Score/eval.json
```

🔄 NOW: What If Something Changes?

Case 1 Code changes

Auto

```
src/model_building.py
```

👉 DVC will:

- rerun **model_building**
- rerun **model_evaluation**
- NOT rerun data stages

Case 2 Raw data changes

Auto

```
data/raw/spam.csv
```

👉 DVC will:

- rerun **ALL stages below ingestion**

Case 3 You add a new output file

Example:

Auto

```
reports/confusion_matrix.png
```

👉 ONLY change:

Auto

```
outs:
  - models/model.pkl
  - reports/confusion_matrix.png
```

⚠️ Don't touch `cmd` or `deps`

Case 4 You add hyperparameters

Use `params.yaml`:

Auto

```
train:
  lr: 0.01
  epochs: 10
```

Then in `dvc.yaml`:

Auto

```
params:
  - train.lr
  - train.epochs
```

Now DVC tracks param changes automatically.

🚫 What NOT to Put in `dvc.yaml`

❌ logging config
❌ Python variables
❌ code logic
❌ environment variables

Only:
✓ commands
✓ file dependencies
✓ file outputs

🧠 Golden Rule (EXAM + INTERVIEW)

> **If a file affects execution → put in `deps`**
> **If a file is created → put in `outs`**

✅ What I Recommend You Do Next (IMPORTANT)

1️⃣ Pick **ONE script** (`data_` ingestion.py)
2️⃣ Write its stage in `dvc.yaml`
3️⃣ Run:

Auto

```
dvc repro
```

4️⃣ See what breaks
5️⃣ Fix paths
6️⃣ Then add next stage

🔴 Short, Correct Answer

> You do NOT need to run `dvc add` for things that are already listed in `outs` of `dvc.yaml`.

DVC will track them **automatically** when you run:

bash Copy code

```bash
dvc repro
```

🧠 The Big Rule (MEMORIZE THIS)

There are TWO ways DVC tracks data:

✅ **1. Pipeline outputs (`outs` in `dvc.yaml`)**

➡️ **NO `dvc add` needed**

✅ **2. Standalone data (not produced by pipeline)**

➡️ **USE `dvc add`**

Standalone data is your input data for example data.csv

NO — listing a folder in `dvc.yaml` does NOT track it by itself.

Tracking happens only when:

- the folder is an `out` of a stage AND
- that stage is actually **executed** (`dvc repro`)

🧠 Very Important Distinction (This clears confusion)

`dvc.yaml` is a PLAN

`dvc add` / `dvc repro` is EXECUTION